SHL Assessment Recommender: Approach

I built a chat API that turns a vague hiring need ("we need someone for a leadership role") into a shortlist of real SHL assessments. It asks a clarifying question when it doesn't know enough, updates the shortlist when requirements change, and answers comparison questions using real catalog facts instead of guessing.

At a glance: 377 assessments in the catalog. Search powered by BM25. Llama 3.3 70B (via Groq) as the LLM. Recall@10 of 0.56 on real test conversations. 7 out of 8 behavior checks passed live. All 15 unit tests passing.

How it works

The service is a small FastAPI app with two endpoints, `/health` and `/chat`. It doesn't remember past conversations itself; every request includes the full chat history so far. For each message, it does five things:

1. Reads the whole conversation and turns it into a search query.
2. Searches the 377 assessments and picks the 25 most relevant ones.
3. Sends the conversation, those 25 candidates, and a set of instructions to the LLM.
4. Gets back a clear, structured answer (not plain text I'd have to interpret).
5. Double checks every recommended assessment against the real catalog before sending it back.

That last step matters most. The LLM never gets to write a product name or link on its own. It can only point to an assessment from the list I gave it. If it ever points to something that isn't on that list, I quietly remove it. This makes a made up assessment or broken link impossible, not just unlikely.

How it finds the right assessments

With only 377 assessments, a heavy search system like a vector database felt unnecessary. It would slow down startup on free hosting without really improving accuracy at this size. Instead I used BM25, a well known keyword search method, matching against each assessment's name, description, category, and job level. Product names count extra, so an exact match always wins.

Plain keyword search has two weak spots. Short names like "OPQ" or "GSA" don't score well, and if someone types an exact product name, it should always be found. I added a small extra step that catches both cases directly, so the search never misses an obvious answer.

The search step only builds a list of good candidates. It never picks the final answer. The LLM decides which of those candidates to actually recommend, whether to add something like a cognitive test by default, and how to update the list as the conversation continues.

How the agent makes decisions

On every turn, the LLM returns one structured answer: its reply text, whether the question is even something it should answer, whether it's ready to give a shortlist, which assessments to recommend, and whether the conversation is finished. A few rules are enforced directly in code, not just requested in the prompt, because prompts alone don't always hold up over a long conversation:

- If the agent declines to answer something (like a legal question), the shortlist is cleared for that reply, even if one was agreed on earlier. I found this by reading SHL's own sample conversations closely: after declining an off topic question, the agent brings the shortlist back once the user confirms they still want it. Getting this timing right turned out to matter a lot.
- The conversation is capped at 8 turns total, so about 4 exchanges. The agent is told to ask at most one or two clarifying questions, then commit to a shortlist using reasonable defaults. Early testing showed it would otherwise keep asking questions past the limit, which hurt accuracy no matter how good the search was.
- Comparison questions are answered only using facts from the catalog, never from general knowledge the model might already have about a product.

What didn't work at first

- **It recommended too quickly on vague messages.** I tested it live with the exact same opening line from SHL's own example, "we need a solution for senior leadership." The agent skipped the clarifying question and guessed a shortlist right away. I updated the instructions to separate two cases: a message that only names a role (should ask first) versus one that names a specific skill, tool, or clear priority (fine to act on immediately). After the fix, both cases worked correctly.
- **One LLM option quietly gave empty answers.** Before settling on Groq, I tried Google's Gemini model. It spends part of its reply budget on invisible "thinking" text before writing the real answer, so with a normal limit it used up the whole budget thinking and never wrote a reply. A single setting change fixed this.
- **Free usage limits shaped some design choices.** My first version sent 40 candidate assessments to the LLM on every turn, which used a lot of tokens and quickly hit rate limits during testing. I trimmed it to 25 candidates and shortened the format for each one, cutting the size roughly in half with no drop in accuracy.
- **Error handling treated every failure the same way at first.** If the LLM gave back badly formatted text, I'd retry automatically. That helps with a formatting mistake, but not with a rate limit error, which won't fix itself in time. Now formatting problems get one retry, and connection or rate limit problems fail straight to a safe, valid response instead of wasting time.

How I tested it

I checked four things, each backed by a script or test rather than a guess:

- **Search quality.** Unit tests confirm the search finds relevant assessments for a query, and correctly matches short names and exact product names.
- **Recommendation quality.** I replayed SHL's 10 sample conversations and checked how many of the expected assessments showed up in the results. Average score: **0.56 out of 1.0**, and no conversation came back completely empty. SHL's expected answers are often more specific than a first attempt would guess, so getting partial credit almost everywhere is a realistic result for a system with no manual tuning per scenario.
- **No made up answers.** This is guaranteed in the code itself, not just hoped for. A test feeds the agent a fake response that tries to recommend something outside the list, and confirms it gets blocked.
- **Overall behavior.** I wrote 8 small test conversations covering exactly what the assignment asks for: refusing off topic questions, refusing legal questions, refusing prompt injection attempts, not recommending on a vague first message, committing when given real detail, updating the list when a constraint changes, and always returning a valid response even on empty input. **7 out of 8 passed.** The one exception was the agent correctly choosing not to add a duplicate test that the shortlist already covered, which was a stricter expectation in my test than the actual correct behavior.

One honest note: daily usage limits meant I couldn't run the full 10 conversation test in a single clean pass. A few runs were interrupted by the provider itself, not by any mistake in the agent. The scripts detect this and mark those runs as inconclusive instead of counting them as failures, so the numbers above reflect real results, not guesses.

AI tools used

Yes. I used an AI coding assistant throughout this project, both to write the code (search, agent logic, tests, evaluation scripts) and to carefully read SHL's assignment document, catalog data, and sample conversations to figure out expected behaviors that weren't written down explicitly, like the rule about clearing the shortlist during a refusal. I also tested the deployed agent against multiple LLM providers (Gemini, then Groq's Llama 3.3) before settling on Groq for reliability, as described above. Every choice described in this document was a decision I made on purpose, not something left on default.